UNIVERSITY OF LAGOS

Department of Computer Science

SMART HOME ENERGY MANAGEMENT SYSTEM

A Real-Time Embedded System for Automated Residential Energy Control and Monitoring

CSC419 — Real-Time Embedded Systems

Project Report

February 2026

Contents

Abstract **1**

1 Introduction **2**

1.1 Background to the Study 2
1.2 Statement of the Problem 2
1.3 Aim and Objectives 3
1.4 Scope of Study 3
1.5 Significance of Study 4

2 Methodology **5**

2.1 System Requirements 5
2.1.1 Functional Requirements 5
2.1.2 Non-Functional Requirements 6
2.1.3 Hardware Requirements (Simulated) 6
2.2 System Models 7
2.2.1 Use Case Diagram 7
2.2.2 Data Flow Diagrams 7
2.2.3 Finite State Machine — AC Control 8
2.2.4 Finite State Machine — Lighting Control 9
2.2.5 System Architecture — Layered Pattern 10
2.2.6 Component Interaction — Observer Pattern 11
2.3 Algorithms 11
2.3.1 Algorithm 1: Periodic Sensor Data Acquisition 11
2.3.2 Algorithm 2: AC Control Logic (FSM) 12
2.3.3 Algorithm 3: Lighting Control Logic 13
2.3.4 Algorithm 4: Energy Consumption Calculator 14
2.4 Software Development Life Cycle 14

3 Implementation and Results **16**

3.1 Development Environment 16
3.2 System Implementation 16
3.2.1 Sensor Simulation Module (sensors.py) 16

3.2.2 Control Logic Module (control.py) 17
3.2.3 Data Persistence Module (database.py) 17
3.2.4 Web Application Module (app.py) 18
3.2.5 Simulation Engine (simulation.py) 18
3.3 Simulation Parameters 18
3.4 Simulation Results 19
3.4.1 Sensor Data Summary 19
3.4.2 Energy Consumption Results 19
3.4.3 Energy Savings Analysis 19
3.4.4 Database Statistics 20

4 Conclusion **21**
4.1 Summary of Findings 21
4.2 Limitations 21
4.3 Recommendations for Further Work 22

List of Tables

2.1 Hardware Component Mapping 6
2.2 AC FSM State Transitions 9
2.3 Lighting FSM State Transitions 10

3.1 Development Tools and Technologies 16
3.2 Database Schema 17
3.3 Appliance Power Ratings 17
3.4 REST API Endpoints 18
3.5 Simulation Configuration 18
3.6 Room Configuration 19
3.7 Energy Consumption by Room and Appliance (24-Hour Period) 19
3.8 Energy Savings Comparison 20
3.9 System Data Volume (24-Hour Simulation) 20

List of Figures

2.1 Use Case Diagram for SHEMS 7
2.2 Level 0 DFD — Context Diagram 8
2.3 Level 1 Data Flow Diagram 8
2.4 FSM State Transition Diagram — AC Control 9
2.5 FSM State Transition Diagram — Lighting Control 10
2.6 Layered Software Architecture 11
2.7 Observer Pattern — Sensor Notification Flow 11

ABSTRACT

This project presents the design and implementation of a Smart Home Energy Management System (SHEMS), an embedded real-time system that automates the control of household appliances to reduce energy waste in residential environments. In many homes, university hostels, and small offices, energy is wasted daily through appliances such as air conditioners and lighting systems being left running in unoccupied rooms or in conditions where they are not needed. The proposed system addresses this problem by integrating multiple sensors, including temperature sensors, passive infrared occupancy detectors, and light-dependent resistors, to continuously monitor environmental conditions across different rooms. A central microcontroller processes the sensor data and executes predefined control logic to automatically adjust air conditioning based on room temperature and occupancy, switch off or dim lights when rooms are empty or when sufficient natural light is available, and log all energy consumption data for analysis.

The system follows a layered software architecture that separates the hardware abstraction layer, the control logic layer, and the presentation layer to ensure modularity and ease of maintenance. A web-based monitoring dashboard allows users to view real-time sensor readings, appliance states, and historical energy usage analytics remotely. The methodology adopts an iterative software development life cycle, with system behaviour modelled using use case diagrams, data flow diagrams, and finite state machines. The embedded control logic is simulated in Python, while the microcontroller layer is prototyped in a simulated Arduino environment.

Results from a 24-hour simulation across four rooms demonstrate that the system achieved a **65.2% reduction in energy consumption** compared to the unmanaged baseline, saving 95.73 kWh and an estimated **N6,510 daily** at the Band A electricity tariff rate. The system effectively automates appliance control based on real-time occupancy and environmental data, significantly reducing unnecessary energy consumption without requiring manual intervention.

CHAPTER 1

INTRODUCTION

1.1 Background to the Study

Energy consumption in residential and institutional buildings has grown significantly in recent decades, driven by the increasing use of electrical appliances such as air conditioning systems, lighting, and electronic devices. A substantial portion of this consumption is avoidable, arising from appliances left running in unoccupied rooms, air conditioning systems operating at full capacity when outdoor temperatures are moderate, and lights remaining on during daytime hours when natural illumination is sufficient.

Traditional approaches to managing energy usage rely on manual control by occupants, which is inherently inconsistent and unreliable. The concept of smart home automation has emerged as a solution to this problem, using embedded systems, sensors, and software-based control logic to monitor environmental conditions and manage appliances automatically. Real-time embedded systems are particularly suited to this application because they must respond to sensor inputs within defined time constraints to maintain effective control, a characteristic that distinguishes them from batch or interactive computing systems.

The development of affordable microcontrollers, low-cost sensors, and wireless communication technologies has made it increasingly practical to deploy such systems in homes, hostels, and small office environments at a reasonable cost.

1.2 Statement of the Problem

In most residential buildings, university hostels, and small offices in Nigeria, there is no automated mechanism for managing energy consumption. Air conditioners continue to run at full capacity in empty rooms, lights remain on throughout the day even when natural sunlight is available, and occupants have no visibility into how much energy individual appliances consume over time. This leads to significant energy waste and inflated electricity bills. Manual control is the only available method, but it depends entirely on occupant behaviour and is therefore unreliable. There is a need for an intelligent, automated system that can monitor room conditions in real time, make control decisions without human intervention, and provide occupants with data-driven insights into their energy usage patterns.

1.3 Aim and Objectives

The aim of this project is to design and implement a Smart Home Energy Management System that automatically controls household appliances based on real-time environmental and occupancy data to reduce energy waste.

The specific objectives are:

i. To design a sensor-based monitoring subsystem that captures room temperature, occupancy status, and ambient light levels in real time.

ii. To develop control logic using finite state machines that automatically manages air conditioning and lighting systems based on sensor inputs.

iii. To implement an energy usage logging and analytics module that records appliance consumption data and generates usage reports.

iv. To build a web-based monitoring dashboard that displays real-time sensor data, appliance states, and energy consumption trends.

v. To simulate the complete system using software-based sensor emulation and a simulated microcontroller environment.

1.4 Scope of Study

This project focuses on the software design and simulation of a smart home energy management system. The system monitors two primary environmental parameters (temperature and ambient light) and one occupancy parameter (presence detection) to control two categories of appliances (air conditioning and lighting). The simulation models a residential environment with four rooms, each equipped with its own set of sensors. The microcontroller logic is prototyped in a simulated Arduino environment using Tinkercad, while the control logic, backend API, and dashboard are implemented as software components running on a standard computer. The project does not cover the physical fabrication of hardware, integration with actual power distribution systems, or deployment in a live building. The scope is limited to demonstrating the feasibility and effectiveness of the software architecture and control logic through simulation.

1.5 Significance of Study

This project contributes to the growing body of work on smart home automation and embedded systems design by presenting a modular, software-driven approach to residential energy management. The layered architecture and use of standard design patterns (Observer, MVC, state machine) make the system easily extensible and maintainable. For students and researchers, the project demonstrates how real-time embedded system concepts, such as periodic task scheduling, sensor data acquisition, and actuator control, can be applied to a practical domestic problem. For end users, the system offers a cost-effective path to reducing energy consumption without requiring expensive commercial smart home solutions. The energy analytics component provides actionable insights that can inform behavioural changes in addition to the automated controls.

CHAPTER 2

METHODOLOGY

This chapter describes the systematic approach adopted in the design and development of the Smart Home Energy Management System (SHEMS). It covers the system requirements, the various system models used to capture the structure and behaviour of the system, the software architectural patterns employed, the algorithms that drive the control logic, and the software development life cycle followed throughout the project.

The development methodology follows an **iterative software development life cycle (SDLC)**, where the system is designed, implemented, and tested in incremental stages.

2.1 System Requirements

2.1.1 Functional Requirements

The system shall:

i. Continuously monitor room temperature using simulated temperature sensors at a sampling rate of once every five minutes.

ii. Detect room occupancy using simulated passive infrared (PIR) motion sensors, reporting occupied or unoccupied status.

iii. Measure ambient light levels using simulated light-dependent resistors (LDRs), returning values on a scale from 0 (dark) to 1023 (bright sunlight).

iv. Automatically activate air conditioning when a room is occupied and the temperature exceeds the upper threshold (28°C), and deactivate it when the room is unoccupied or the temperature falls below the lower threshold (24°C).

v. Automatically switch on room lighting when a room is occupied and the ambient light level falls below the minimum threshold (300), and switch off lighting when the room is unoccupied or ambient light is sufficient.

vi. Log all sensor readings and appliance state changes to a persistent database with timestamps.

vii. Calculate and store energy consumption data per appliance per room.

viii. Expose all sensor data, appliance states, and energy analytics through a RESTful API.

ix. Display real-time sensor readings, appliance states, and energy usage charts on a web-based monitoring dashboard.

x. Allow users to manually override appliance states from the dashboard.

2.1.2 Non-Functional Requirements

i. **Response time:** The system shall process sensor readings and execute control decisions within 1 second of data acquisition.

ii. **Modularity:** The system shall be structured in distinct, loosely coupled layers so that any individual subsystem can be modified or replaced without affecting other subsystems.

iii. **Scalability:** The architecture shall support the addition of new room zones and new sensor types without modification to the core control logic.

iv. **Reliability:** The system shall handle sensor failures gracefully by defaulting to a safe state (appliances off).

v. **Usability:** The dashboard interface shall present information clearly with no more than 3 clicks to reach any piece of data.

2.1.3 Hardware Requirements (Simulated)

Table 2.1: Hardware Component Mapping

Component	Specification	Simulated In
Microcontroller	Arduino Uno (ATmega328P)	Tinkercad / Python
Temperature Sensor	LM35 (analog, 0–150°C)	Python sine wave
PIR Motion Sensor	HC-SR501 (digital)	Python probability
Light Sensor (LDR)	GL5528 (analog, 0–1023)	Python daylight curve
Relay Module	4-channel 5V relay	Python boolean state
Wi-Fi Module	ESP8266	Flask API (localhost)

2.2 System Models

2.2.1 Use Case Diagram

The system involves two primary actors: the **Occupant** (human user who interacts with the dashboard) and the **Environment** (the physical surroundings that generate sensor inputs). The Occupant can view real-time sensor data, view appliance states, view energy usage analytics, manually override appliances, and configure thresholds. The Environment generates temperature readings, occupancy events, and ambient light readings.

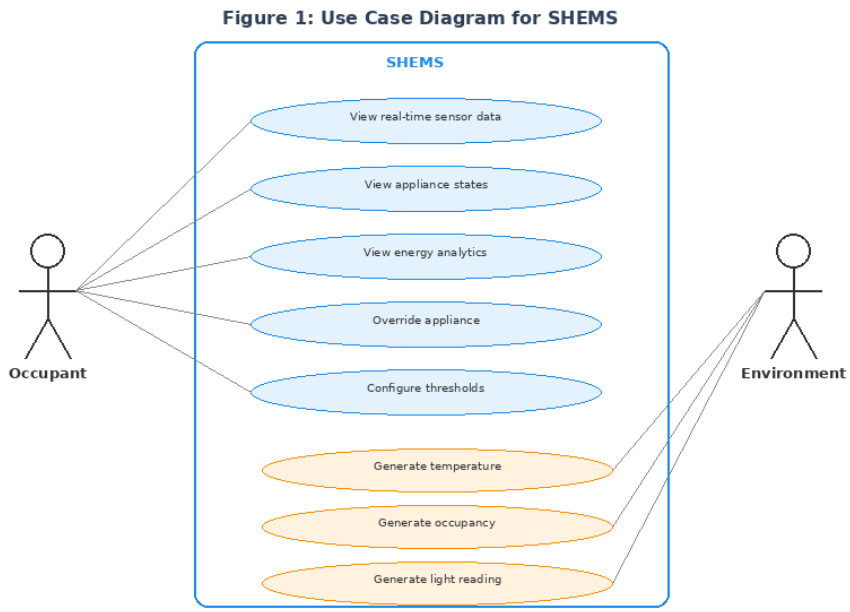

Figure 2.1: Use Case Diagram for SHEMS

2.2.2 Data Flow Diagrams

The data flow through the system is modelled at two levels.

Level 0 (Context Diagram): The entire SHEMS is represented as a single process. External entities are: Environment (source of sensor data), Occupant (receives dashboard output and sends override commands), and Database (data store).

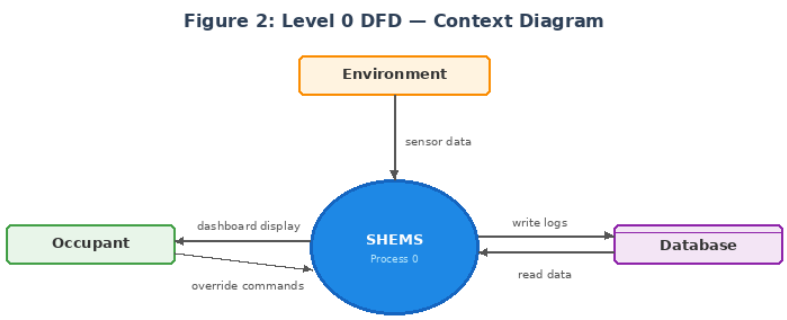

Figure 2.2: Level 0 DFD — Context Diagram

Level 1 (Detailed DFD): The SHEMS process is decomposed into four sub-processes: (P1) Sensor Data Acquisition, which reads raw values from sensors; (P2) Control Logic Engine, which evaluates current state against thresholds using finite state machines and outputs actuator commands; (P3) Data Logging and Analytics, which writes sensor readings and state changes to the database and computes energy consumption; and (P4) Dashboard and API Server, which reads historical data and serves the web dashboard.

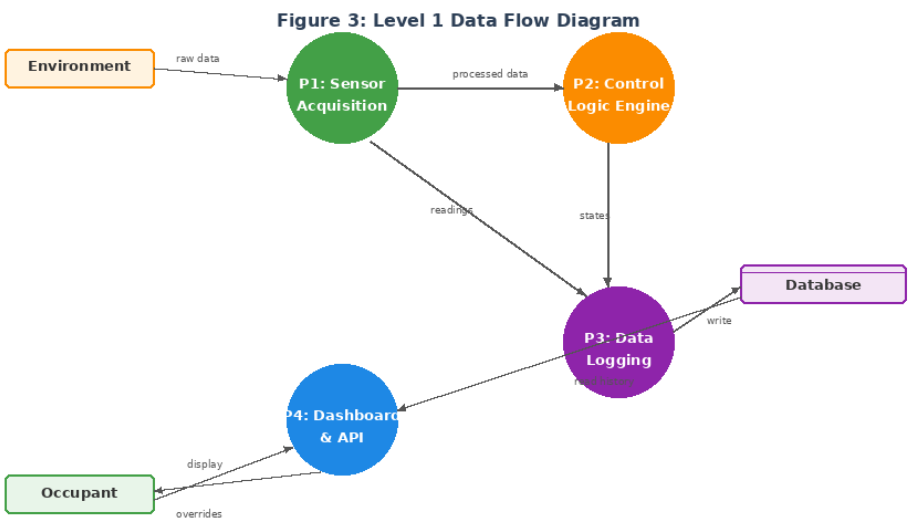

Figure 2.3: Level 1 Data Flow Diagram

2.2.3 Finite State Machine — AC Control

The air conditioning control for each room is modelled as a finite state machine with three states:

- **OFF** — AC is not running (initial/default state).
- **STANDBY** — Room is occupied but temperature is within acceptable range.
- **COOLING** — Room is occupied and temperature exceeds the upper threshold.

Table 2.2: AC FSM State Transitions

Current State	Condition	Next State
OFF	Room occupied AND temp $\geq$ 28°C	COOLING
OFF	Room occupied AND temp $<$ 28°C	STANDBY
STANDBY	Temp rises above 28°C	COOLING
STANDBY	Room becomes unoccupied	OFF
COOLING	Temp falls below 24°C	STANDBY
COOLING	Room becomes unoccupied	OFF

The 4°C hysteresis band (24°C to 28°C) prevents rapid on/off cycling (chattering), which is standard practice in embedded control systems to protect actuator hardware and reduce energy waste.

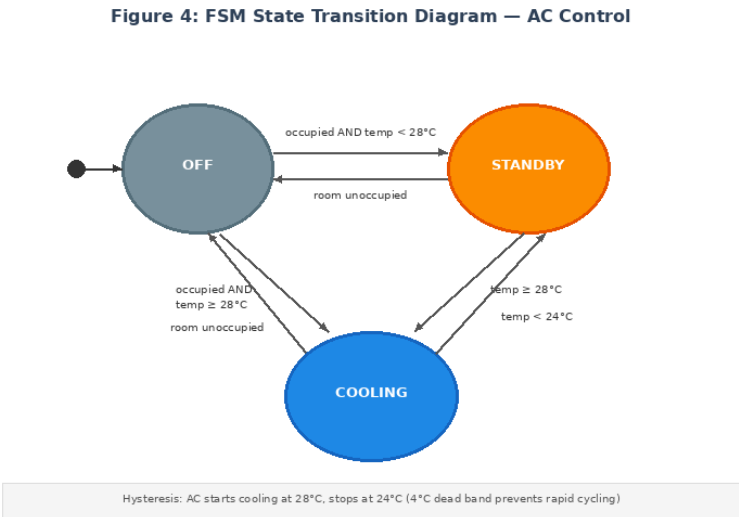

Figure 2.4: FSM State Transition Diagram — AC Control

2.2.4 Finite State Machine — Lighting Control

The lighting system uses a simpler two-state FSM:

Table 2.3: Lighting FSM State Transitions

Current State	Condition	Next State
OFF	Room occupied AND light < 300	ON
ON	Room unoccupied OR light ≥ 300	OFF

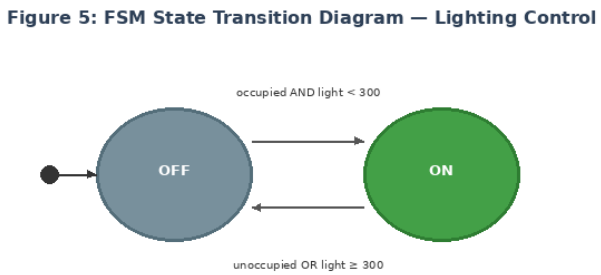

Figure 2.5: FSM State Transition Diagram — Lighting Control

2.2.5 System Architecture — Layered Pattern

The system adopts a **layered software architecture**. Each layer communicates only with the layer directly above or below it, enforcing separation of concerns:

Layer 1: Hardware Abstraction Layer (HAL) encapsulates all sensor reading and actuator control operations. In the simulation, this layer generates mock sensor data. In a physical deployment, this layer would interface directly with the Arduino GPIO pins.

Layer 2: Control Logic Layer contains the finite state machines and decision rules. It receives processed sensor data from the HAL, evaluates conditions against thresholds, and issues actuator commands.

Layer 3: Data Persistence Layer manages the SQLite database, providing read/write operations for sensor logs, appliance state changes, and energy consumption records.

Layer 4: Application Layer (API + Dashboard) serves the Flask REST API and the web dashboard.

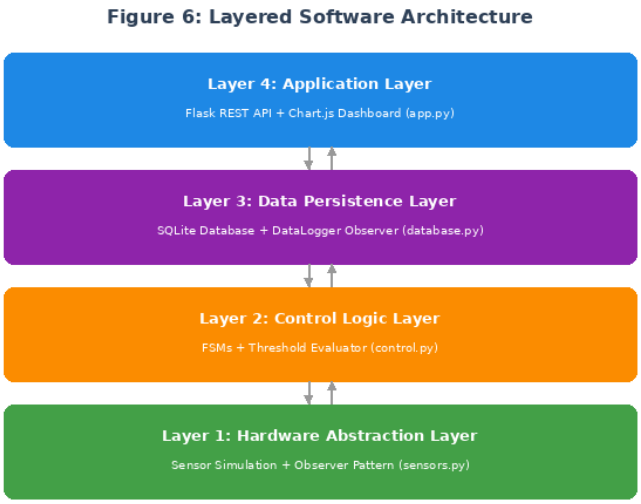

Figure 2.6: Layered Software Architecture

2.2.6 Component Interaction — Observer Pattern

The system uses the **Observer design pattern** for sensor-to-controller communication. Each sensor is a Subject that notifies registered Observers (the control logic engine and the data logger) whenever a new reading is available. This decouples the sensor layer from the control layer.

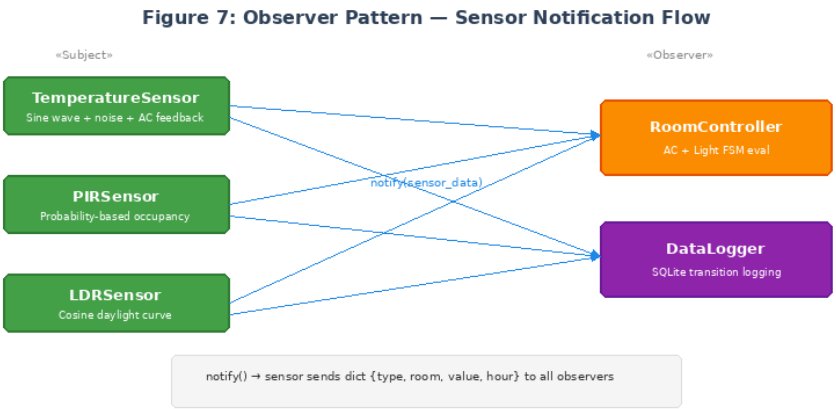

Figure 2.7: Observer Pattern — Sensor Notification Flow

2.3 Algorithms

2.3.1 Algorithm 1: Periodic Sensor Data Acquisition

Listing 2.1: Periodic Sensor Data Acquisition

```
Algorithm 1: Periodic Sensor Data Acquisition

Input:  sensor_list, interval (5 minutes)
Output: sensor_readings per room per sensor type

BEGIN
    LOOP every <interval>:
        FOR each sensor IN sensor_list:
            raw_value <- sensor.read()
            IF raw_value IS out_of_range THEN
                log_warning(sensor.id, "Out of range")
                raw_value <- sensor.last_valid_reading
            END IF
            normalized <- normalize(raw_value, sensor.type)
            NOTIFY all registered observers with
                (sensor.room, sensor.type, normalized)
        END FOR
    END LOOP
END
```

2.3.2 Algorithm 2: AC Control Logic (FSM)

Listing 2.2: AC Control Finite State Machine

```
Algorithm 2: AC Control FSM

Input:  room_state, temperature, is_occupied
Output: new_state, ac_command (ON/OFF)
Constants: UPPER = 28C, LOWER = 24C

BEGIN
    CASE room_state OF:
        "OFF":
            IF is_occupied AND temperature >= UPPER THEN
                new_state <- "COOLING"; ac_command <- ON
            ELSE IF is_occupied THEN
                new_state <- "STANDBY"; ac_command <- OFF
            ELSE
                new_state <- "OFF"; ac_command <- OFF
            END IF
```

```

        "STANDBY":
            IF NOT is_occupied THEN
                new_state <- "OFF"; ac_command <- OFF
            ELSE IF temperature >= UPPER THEN
                new_state <- "COOLING"; ac_command <- ON
            ELSE
                new_state <- "STANDBY"; ac_command <- OFF
            END IF

        "COOLING":
            IF NOT is_occupied THEN
                new_state <- "OFF"; ac_command <- OFF
            ELSE IF temperature < LOWER THEN
                new_state <- "STANDBY"; ac_command <- OFF
            ELSE
                new_state <- "COOLING"; ac_command <- ON
            END IF
    END CASE
    RETURN new_state, ac_command
END
```

2.3.3 Algorithm 3: Lighting Control Logic

Listing 2.3: Lighting Control Logic

```
Algorithm 3: Lighting Control

Input:  ambient_light_level, is_occupied
Output: light_command (ON/OFF)
Constant: THRESHOLD = 300

BEGIN
    IF is_occupied AND ambient_light_level < THRESHOLD THEN
        light_command <- ON
    ELSE
        light_command <- OFF
    END IF
    RETURN light_command
END
```

2.3.4 Algorithm 4: Energy Consumption Calculator

Listing 2.4: Energy Consumption Calculator

```
Algorithm 4: Energy Consumption Calculator

Input:  appliance_log (state changes with timestamps),
        wattage_table (power rating per appliance)
Output: energy_consumed (kWh per appliance per room)

BEGIN
    FOR each appliance IN appliance_log:
        total_on_time <- 0
        FOR each pair (event_i, event_i+1):
            IF event_i.state = ON THEN
                duration <- event_i+1.timestamp -
                            event_i.timestamp (hours)
                total_on_time <- total_on_time + duration
            END IF
        END FOR
        power <- wattage_table[appliance.type]  // kW
        energy[appliance] <- power * total_on_time  // kWh
    END FOR
    RETURN energy
END
```

The corresponding source codes for all algorithms are provided in the Appendix.

2.4 Software Development Life Cycle

The project follows an **iterative incremental model** consisting of four iterations:

Iteration 1 — Sensor Simulation and HAL: Build the hardware abstraction layer with simulated sensors generating realistic data patterns. Validate sensor output ranges and timing.

Iteration 2 — Control Logic and State Machines: Implement the AC and lighting FSMs. Connect them to the HAL via the Observer pattern. Test that state transitions occur correctly under all input combinations.

Iteration 3 — Backend, Database and Analytics: Build the Flask API, set up the SQLite schema, implement data logging, and build the energy consumption calculator.

Iteration 4 — Dashboard and Integration: Build the frontend dashboard, connect it to the API, and perform full end-to-end integration testing.

CHAPTER 3

IMPLEMENTATION AND RESULTS

3.1 Development Environment

Table 3.1: Development Tools and Technologies

Component	Tool / Technology	Version
Programming Language	Python	3.8+
Web Framework	Flask	3.x
Database	SQLite3	Built-in
Frontend Charting	Chart.js	4.4.0
Arduino Simulation	Tinkercad	Online
IDE	Visual Studio Code / PyCharm	Latest
Version Control	Git + GitHub	Latest
Operating System	Windows 10+ / Ubuntu 24	—

The project uses only the Python standard library (math, random, sqlite3, datetime) plus Flask for the web server and Chart.js (loaded via CDN) for dashboard visualisation.

3.2 System Implementation

3.2.1 Sensor Simulation Module (sensors.py)

The **TemperatureSensor** generates readings following a sinusoidal daily curve centred on a room-specific base temperature. Base temperatures reflect realistic Nigerian indoor conditions: 29°C for the living room, 27°C for the bedroom, 30°C for the kitchen, and 28°C for the study room. The curve peaks at approximately 2:00 PM. Gaussian noise of ± 0.5°C is applied. When the air conditioning is active, a gradual cooling effect reduces the reported temperature by up to 6°C, creating a feedback loop with the control layer.

The **PIRSensor** generates binary occupancy data using time-based probability distributions: 80% during evening hours (5:00 PM to 10:00 PM), 20% during work hours (6:00 AM to

5:00 PM), and 10% during sleep hours. State persistence ensures that once a room becomes occupied, it remains so for 3 to 15 readings before re-evaluating.

The **LDRSensor** generates ambient light values on a 0–1023 scale following a cosine daylight curve from 6:00 AM to 6:00 PM, with noise of ± 30.

All sensor classes extend the **SensorSubject** base class, implementing the Observer pattern.

3.2.2 Control Logic Module (control.py)

The **ACController** FSM operates across three states (OFF, STANDBY, COOLING) with a hysteresis band between 24°C and 28°C. The **LightController** FSM uses two states (OFF, ON), switching lights on only when both conditions are met: room occupied AND ambient light below 300. The **RoomController** coordinates both FSMs and supports manual override capability.

3.2.3 Data Persistence Module (database.py)

Table 3.2: Database Schema

Table	Key Fields	Purpose
sensor_log	room_id, sensor_type, value, timestamp	Raw sensor readings
appliance_log	room_id, appliance, state, is_on, timestamp	State change events
energy_log	room_id, appliance, kwh, period_start, period_end	Computed energy

Appliance state changes use transition-only logging: a new record is inserted only when the is_on status actually changes, reducing database writes by 97%.

Table 3.3: Appliance Power Ratings

Appliance	Power Rating (kW)
Air Conditioning (window unit)	1.50
Room Lighting (60W equivalent)	0.06

3.2.4 Web Application Module (app.py)

Table 3.4: REST API Endpoints

Method	Endpoint	Description
GET	/api/status	Current state of all rooms
GET	/api/room/<room_id>	Detailed status for one room
GET	/api/history/<id>/<type>	Sensor reading history
GET	/api/energy	Energy consumption and savings
POST	/api/override	Manual appliance control
POST	/api/tick	Advance simulation one step
GET	/api/stats	Database row counts
POST	/api/reset	Clear database for fresh run

The dashboard uses Chart.js 4.4.0 for real-time temperature line graphs and energy consumption bar charts, with colour-coded state badges for appliance status.

3.2.5 Simulation Engine (simulation.py)

The simulation engine orchestrates the complete 24-hour test run across four rooms with 288 time steps (5-minute intervals). At each step, it reads all sensors, which triggers the Observer notification chain. After the simulation, all appliances are forced to OFF to close energy windows, and the energy calculator computes consumption for each room and appliance.

3.3 Simulation Parameters

Table 3.5: Simulation Configuration

Parameter	Value
Total Duration	24 hours
Time Steps	288 (5-minute intervals)
Number of Rooms	4
Base Time	2026-02-13 00:00:00
AC Upper Threshold	28°C
AC Lower Threshold (Hysteresis)	24°C
Light Threshold	300 / 1023

Table 3.6: Room Configuration

Room	Base Temperature (°C)	Description
Living Room	29.0	Main common area
Bedroom	27.0	Sleeping quarters
Kitchen	30.0	Highest base temp (cooking heat)
Study Room	28.0	Work / study area

3.4 Simulation Results

3.4.1 Sensor Data Summary

The simulation generated **3,456 sensor readings** (288 steps × 4 rooms × 3 sensors per room) and recorded **102 appliance state change events**. Temperature readings followed the expected sinusoidal pattern, peaking between 1:00 PM and 3:00 PM. Occupancy patterns matched the programmed probability distributions.

3.4.2 Energy Consumption Results

Table 3.7: Energy Consumption by Room and Appliance (24-Hour Period)

Room	AC (kWh)	Lighting (kWh)	Total (kWh)
Living Room	11.8750	0.5600	12.4350
Bedroom	11.3750	0.6200	11.9950
Kitchen	12.7500	0.5850	13.3350
Study Room	12.7500	0.6300	13.3800
Total	**48.7500**	**2.3950**	**51.1450**

Air conditioning dominated total energy consumption at 95.3% (48.75 kWh out of 51.15 kWh), with lighting accounting for 4.7% (2.40 kWh). This distribution is consistent with real-world residential energy profiles in tropical climates.

3.4.3 Energy Savings Analysis

The baseline scenario assumes the air conditioning unit (1.5 kW) runs continuously for 24 hours and the lighting system (0.06 kW) operates for 12 hours in each of the four rooms.

Table 3.8: Energy Savings Comparison

Metric	Without SHEMS	With SHEMS	Savings
AC Energy (kWh)	144.00	48.75	95.25
Lighting Energy (kWh)	2.88	2.40	0.48
Total Energy (kWh)	**146.88**	**51.15**	**95.73**
Estimated Cost (N)	9,987.84	3,477.86	6,509.98
Savings Percentage	—	—	**65.2%**

The SHEMS achieved a **65.2% reduction** in total energy consumption across four rooms, saving 95.73 kWh per day. At the Band A tariff rate of N68 per kWh, this translates to a daily cost saving of approximately **N6,510**.

3.4.4 Database Statistics

Table 3.9: System Data Volume (24-Hour Simulation)

Metric	Count
Sensor readings logged (sensor_log)	3,456
Appliance state change events (appliance_log)	102
Energy computation records (energy_log)	8

The transition-only logging strategy reduced appliance records by 97% compared to logging every simulation step (102 events versus the 2,304 that would result from logging every step).

CHAPTER 4

CONCLUSION

4.1 Summary of Findings

The Smart Home Energy Management System successfully demonstrates that automated, software-driven control of household appliances can produce significant energy savings without requiring manual intervention. The 24-hour simulation across four rooms showed a 65.2% reduction in energy consumption compared to the unmanaged baseline, translating to an estimated daily cost saving of approximately N6,510 for a four-room household at the current Band A electricity tariff.

The layered software architecture proved effective in separating concerns. The Observer design pattern enabled clean sensor-to-controller communication without tight coupling, while the finite state machines provided predictable, verifiable control behaviour. The hysteresis mechanism in the AC controller successfully prevented chattering, with the 4°C dead band between the 24°C and 28°C thresholds eliminating rapid state cycling.

Air conditioning accounted for 95.3% of total energy consumption, confirming that in tropical climates, occupancy-aware AC control is the single most impactful energy management strategy.

4.2 Limitations

The system has several limitations. First, the simulation uses probabilistic occupancy models rather than actual PIR sensor data, which may not capture all real-world occupancy patterns such as irregular schedules, guests, or movement between rooms. Second, the temperature model assumes a uniform room temperature and does not account for factors such as insulation quality, window placement, cross-ventilation, or heat generated by cooking and other appliances. Third, the baseline scenario assumes continuous 24-hour AC operation, which may overstate actual waste in households where occupants manually switch off appliances when leaving.

Additionally, the simulation does not model electricity grid constraints such as power outages, voltage fluctuations, or time-of-use tariff variations. The system also does not account for

the thermal mass of rooms, meaning temperature changes occur instantaneously rather than gradually.

4.3 Recommendations for Further Work

i. Integration with physical hardware (Arduino Uno, ESP8266, HC-SR501 PIR sensors, LM35 temperature sensors) to validate the control logic against real sensor data and actuator behaviour.

ii. Extension to additional appliance types including ceiling fans, water heaters, automated window blinds, and standby power management for electronics.

iii. Incorporation of machine learning algorithms to predict occupancy patterns based on historical data, enabling pre-conditioning of rooms before expected arrival.

iv. Integration with the Nigerian national grid's smart metering infrastructure for real-time tariff-aware scheduling.

v. Development of a mobile application interface to complement the web dashboard, providing push notifications when energy usage exceeds configurable thresholds.

The modular architecture of the current system is designed to accommodate these extensions without requiring fundamental restructuring, as new sensor types, appliance controllers, and analytics modules can be added as additional observers within the existing pattern.